Local Completeness Logic on strategic term rewriting

Candidate: Niccolò Piazzesi *Supervisor*: Roberto Bruni

Examiner: Francesco Gavazzo

December 2, 2022

Università di Pisa

Outline

1. Introduction
2. The language Strat
3. Local Completeness Logic on Strat
4. An example
5. Conclusions

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

An example

$$\Sigma = \{0, succ, add\} \qquad L = \{\text{addZero}, \text{addSucc}\}$$

$$R = \{\text{addZero: add(0, x)} \rightarrow \text{x}, \quad \text{addSucc: add(succ(x), y)} \rightarrow \text{succ(add(x, y))}\}$$

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

An example

$$\Sigma = \{0, succ, add\} \qquad L = \{\text{addZero}, \text{addSucc}\}$$

$$R = \{\text{addZero: add(0, x)} \rightarrow \text{x}, \quad \text{addSucc: add(succ(x), y)} \rightarrow \text{succ(add(x, y))}\}$$

add(succ(0), add(0, succ(succ(0))))

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

An example

$$\Sigma = \{0, succ, add\} \qquad L = \{\text{addZero}, \text{addSucc}\}$$

$R = \{$addZero: add(0, x) $\rightarrow$ x, **addSucc add(succ(x), y) $\rightarrow$ succ(add(x, y))**$\}$

$$\textbf{add(succ(0), add(0, succ(succ(0))))} \overset{\text{addSucc}}{\Rightarrow}$$

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

An example

$$\Sigma = \{0, succ, add\} \qquad L = \{\text{addZero}, \text{addSucc}\}$$

$$R = \{\text{addZero: add(0, x)} \rightarrow \text{x}, \quad \text{addSucc: add(succ(x), y)} \rightarrow \text{succ(add(x, y))}\}$$

$$\text{add(succ(0), add(0, succ(succ(0))))} \overset{\text{addSucc}}{\Rightarrow} \text{succ(add(0,add(0,succ(succ(0)))))}$$

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

An example

$$\Sigma = \{0, succ, add\} \qquad L = \{\text{addZero}, \text{addSucc}\}$$

$$R = \{\textbf{addZero: add(0, x)} \rightarrow \textbf{x}\ , \quad \text{addSucc: add(succ(x), y)} \rightarrow \text{succ(add(x, y))}\}$$

$$\text{add(succ(0), add(0, succ(succ(0))))} \overset{\text{addSucc}}{\Rightarrow} \text{succ(add(0,}\textbf{add(0,succ(succ(0)))}\text{))} \overset{\text{addZero}}{\Rightarrow}$$

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

An example

$$\Sigma = \{0, succ, add\} \qquad L = \{\text{addZero}, \text{addSucc}\}$$

$$R = \{\text{addZero: add(0, x)} \rightarrow \text{x}, \quad \text{addSucc: add(succ(x), y)} \rightarrow \text{succ(add(x, y))}\}$$

$$\text{add(succ(0), add(0, succ(succ(0))))} \overset{\text{addSucc}}{\Rightarrow} \text{succ(add(0,add(0,succ(succ(0)))))} \overset{\text{addZero}}{\Rightarrow} \text{succ(add(0,succ(succ(0))))} \cdots$$

Term rewriting systems

Term rewriting is a well-established computational model, where structured objects (**terms**) are transformed by the repeated application of rules (**rewriting**).

$$(\Sigma, R, L)$$

$\Sigma =$ signature of terms $\qquad R =$ set of rules $\qquad L =$ rule labels

An example

$$\Sigma = \{0, succ, add\} \qquad L = \{\text{addZero}, \text{addSucc}\}$$

$$R = \{\text{addZero: add(0, x)} \rightarrow \text{x}, \quad \text{addSucc: add(succ(x), y)} \rightarrow \text{succ(add(x, y))}\}$$

$$\text{add(succ(0), add(0, succ(succ(0))))} \overset{\text{addSucc}}{\Rightarrow} \text{succ(add(0,add(0,succ(succ(0)))))} \overset{\text{addZero}}{\Rightarrow} \text{succ(add(0,succ(succ(0))))} \cdots$$

Rewriting can be **non-terminating** and **non-deterministic**.

Our work

Our goal: prove properties of term rewriting systems using well-known **program verification** methodologies: **Logic and Abstraction**.

Our work

Our goal: prove properties of term rewriting systems using well-known **program verification** methodologies: **Logic and Abstraction**.

Issues:

Our work

Our goal: prove properties of term rewriting systems using well-known **program verification** methodologies: **Logic and Abstraction**.

Issues:

1. **What is a program in term rewriting?**

Our work

Our goal: prove properties of term rewriting systems using well-known **program verification** methodologies: **Logic and Abstraction**.

Issues:

1. **What is a program in term rewriting?** $\rightarrow$ **Strategies**

   A strategy is an algorithm that controls how rewriting rules are applied.

Our work

Our goal: prove properties of term rewriting systems using well-known **program verification** methodologies: **Logic and Abstraction**.

Issues:

1. **What is a program in term rewriting?** $\rightarrow$ **Strategies**

   A strategy is an algorithm that controls how rewriting rules are applied.

2. **What verification framework to use?**

Our work

Our goal: prove properties of term rewriting systems using well-known **program verification** methodologies: **Logic and Abstraction**.

Issues:

1. **What is a program in term rewriting?** $\rightarrow$ **Strategies**

   A strategy is an algorithm that controls how rewriting rules are applied.

2. **What verification framework to use?** $\rightarrow$ **Local Completeness Logic**

   Local Completeness Logic is a proof system capable of reasoning about both the correctness and incorrectness of a program specification (under mild assumptions).

Strategies

- Strategies make rewriting **programmable**. A strategy controls what rules are applied. It can compose rules together, repeatedly apply certain rules and so on.
- A **strategy program** transforms an input term t by applying rules following a strategy expression S.

Strategies

- Strategies make rewriting **programmable**. A strategy controls what rules are applied. It can compose rules together, repeatedly apply certain rules and so on.
- A **strategy program** transforms an input term t by applying rules following a strategy expression S.

Strategy expressions are defined using **strategy languages**.

Strategy languages

Express complex strategies by composing basic primitives.

Strategy languages

Express complex strategies by composing basic primitives.

Many existing languages:

- Stratego [Visser, 2001]
- Maude strategy language [Clavel et al., 2020, Chapter 10]
- Elan [Borovanskỳ and Peter, 1997]
- ...

Strategy languages

Express complex strategies by composing basic primitives.

Many existing languages:

- Stratego [Visser, 2001]
- Maude strategy language [Clavel et al., 2020, Chapter 10]
- Elan [Borovanskỳ and Peter, 1997]
- ...

All languages share some common features and constructs:

- Pattern matching $?t$
- Pattern building $!p$
- Rule application addSucc
- Strategy composition $s_1; s_2$
- Prioritized choice $s_1 <+ s_2$
- Iteration **while** c **do** s

...

The language Strat

Strat is a **store-based** strategy language, parametric to a generic term rewriting system **TRS** $= (\Sigma, R, L)$.

Strat programs modify and update **variables** storing the result of rewriting computations, namely the **terms** of the system.

The language is divided in two main parts:

The language Strat

Strat is a **store-based** strategy language, parametric to a generic term rewriting system **TRS** $= (\Sigma, R, L)$.

Strat programs modify and update **variables** storing the result of rewriting computations, namely the **terms** of the system.

The language is divided in two main parts:

1. **Strategy expressions** transform an input term t to produce a final value t', by controlling the application of rules to it.

The language Strat

Strat is a **store-based** strategy language, parametric to a generic term rewriting system **TRS** $= (\Sigma, R, L)$.

Strat programs modify and update **variables** storing the result of rewriting computations, namely the **terms** of the system.

The language is divided in two main parts:

1. **Strategy expressions** transform an input term t to produce a final value t', by controlling the application of rules to it.
2. **Strategy commands** assign values to variables, and control the flow of execution based on the result of expressions.

Expressions

$$sop@t$$

An expression applies an **operator** op to a term t.
A symbol KO is added to Σ, to represent failure of application.

Expressions

$$sop@t$$

An expression applies an **operator** op to a term t.
A symbol KO is added to Σ, to represent failure of application.

The typical operators are provided:

- matching a term pattern p.

$$?succ(m) \mathbin{@} succ(succ(zero)) \qquad ☺$$
$$?f(x) \mathbin{@} g(c) \qquad KO$$

Expressions

$$sop@t$$

An expression applies an **operator** op to a term t.
A symbol KO is added to Σ, to represent failure of application.

The typical operators are provided:

- matching a term pattern p.

$$?succ(m) \ @ \ succ(succ(zero)) \qquad ☺$$
$$?f(x) \ @ \ g(c) \qquad KO$$

- apply rules.

$$\text{addSucc} \ @ \ add(succ(0), succ(0)) \qquad ☺$$
$$\text{addSucc} \ @ \ add(0, succ(0)) \qquad KO$$

Expressions

$$sop@t$$

An expression applies an **operator** op to a term t.
A symbol KO is added to Σ, to represent failure of application.

The typical operators are provided:

- matching a term pattern p.

$$?succ(m) \mathbin{@} succ(succ(zero)) \quad ☺$$
$$?f(x) \mathbin{@} g(c) \quad KO$$

- apply rules.

addSucc $\mathbin{@}$ $add(succ(0), succ(0))$ ☺
addSucc $\mathbin{@}$ $add(0, succ(0))$ KO

- build a term from current store.

$$a \mapsto succ(0) \qquad b \mapsto 0$$
$$!add(a, b) \mathbin{@} t \longrightarrow add(succ(0), 0)$$

Expressions

$$sop@t$$

An expression applies an **operator** op to a term t.
A symbol KO is added to Σ, to represent failure of application.

The typical operators are provided:

- matching a term pattern p.

  $?succ(m) \mathbin{@} succ(succ(zero))$ ☺
  $?f(x) \mathbin{@} g(c)$ KO

- build a term from current store.

  $a \mapsto succ(0)$ $\quad b \mapsto 0$
  $!add(a, b) \mathbin{@} t \longrightarrow add(succ(0), 0)$

- apply rules.

  addSucc $\mathbin{@}$ $add(succ(0), succ(0))$ ☺
  addSucc $\mathbin{@}$ $add(0, succ(0))$ KO

- choose the first successful operator.

  addSucc <+ **addZero** $\mathbin{@}$ $add(0, succ(0))$
  addSucc <+ addZero $\mathbin{@}$ $add(succ(0), succ(0))$

...

Commands

$\mathbf{x} := \mathbf{sexp}$: evaluate the expression $sexp$ and assign the result to x.

$$\{x \mapsto 0, y \mapsto succ(0)\} \quad \mathbf{x} := !\mathbf{succ}(\mathbf{y}) \quad \{x \mapsto succ(succ(0)), y \mapsto succ(0)\}$$

Commands

$\mathbf{x} := \mathbf{sexp}$: evaluate the expression $sexp$ and assign the result to x.

$$\{x \mapsto 0, y \mapsto succ(0)\} \quad \mathbf{x} := !\mathbf{succ}(\mathbf{y}) \quad \{x \mapsto succ(succ(0)), y \mapsto succ(0)\}$$

sexp <?> p: evaluate the expression $sexp$ and match the result to the pattern p, binding the matching sub-terms in the process.

$$\{x \mapsto succ(0)\} \quad x \text{ <?> } succ(m) \quad \{x \mapsto succ(0), \quad m \mapsto 0\}$$

Commands

$\mathbf{x} := \mathbf{sexp}$: evaluate the expression $sexp$ and assign the result to x.

$$\{x \mapsto 0, y \mapsto succ(0)\} \quad \mathbf{x} := !\mathbf{succ}(\mathbf{y}) \quad \{x \mapsto succ(succ(0)), y \mapsto succ(0)\}$$

sexp <?> p: evaluate the expression $sexp$ and match the result to the pattern p, binding the matching sub-terms in the process.

$$\{x \mapsto succ(0)\} \quad x \text{ <?> } succ(m) \quad \{x \mapsto succ(0), \quad m \mapsto 0\}$$

test sexp: continue execution **only** if **sexp** does not fail.

$$\{x \mapsto f(c)\} \quad \text{test } ?f(y)@x \quad ☺$$
$$\{x \mapsto f(c)\} \quad \text{test } ?g(y)@x \quad ☹$$

Commands

$\mathbf{x} := \mathbf{sexp}$: evaluate the expression $sexp$ and assign the result to x.

$$\{x \mapsto 0, y \mapsto succ(0)\} \quad \mathbf{x} := !\mathbf{succ}(\mathbf{y}) \quad \{x \mapsto succ(succ(0)), y \mapsto succ(0)\}$$

sexp <?> p: evaluate the expression $sexp$ and match the result to the pattern p, binding the matching sub-terms in the process.

$$\{x \mapsto succ(0)\} \quad x \text{ <?> } succ(m) \quad \{x \mapsto succ(0), \quad m \mapsto 0\}$$

test sexp: continue execution **only** if **sexp** does not fail.

$$\{x \mapsto f(c)\} \quad \text{test } ?f(y)@x \quad ☺$$
$$\{x \mapsto f(c)\} \quad \text{test } ?g(y)@x \quad ☹$$

Commands are composed through **regular operators**:

$c_1; c_2$ sequential composition $\qquad$ $c_1 \oplus c_2$ parallel composition

c^* Kleene iteration

Derived command

More familiar constructs can be derived from the primitives.

$$x@ = sop \stackrel{\text{def}}{=} x := sop@x;$$

$$\textbf{if } sexp \textbf{ then } c_1 \textbf{ else } c_2 \stackrel{\text{def}}{=} (\text{test } sexp; c_1) \oplus (\text{test } not(sexp); c_2)$$

$$\textbf{while } sexp \textbf{ do } c \stackrel{\text{def}}{=} (\text{test } sexp; c)^*; \text{test } not(sexp)$$

$$\textbf{case } p \textbf{ of } p_1 \to c_1 | \cdots | p_n \to c_n | \stackrel{\text{def}}{=} (p_1 \text{ <?> } p; c_1) \oplus \cdots \oplus (p_n \text{ <?> } p; c_n)$$

Derived command

More familiar constructs can be derived from the primitives.

$$x@ = sop \stackrel{\text{def}}{=} x := sop@x;$$

$$\textbf{if } sexp \textbf{ then } c_1 \textbf{ else } c_2 \stackrel{\text{def}}{=} (\text{test } sexp; c_1) \oplus (\text{test } not(sexp); c_2)$$

$$\textbf{while } sexp \textbf{ do } c \stackrel{\text{def}}{=} (\text{test } sexp; c)^*; \text{test } not(sexp)$$

$$\textbf{case } p \textbf{ of } p_1 \to c_1 | \cdots | p_n \to c_n | \stackrel{\text{def}}{=} (p_1 \text{ <?> } p; c_1) \oplus \cdots \oplus (p_n \text{ <?> } p; c_n)$$

Higher level strategies are defined using this syntactic sugar:

```
// iteratively applies a strategy s
//to the parameter x until it fails.

norm(s, x) =
while s@x do
x @= s;
```

Derived command

More familiar constructs can be derived from the primitives.

$$x@ = sop \stackrel{\text{def}}{=} x := sop@x;$$

$$\textbf{if } sexp \textbf{ then } c_1 \textbf{ else } c_2 \stackrel{\text{def}}{=} (\text{test } sexp; c_1) \oplus (\text{test } not(sexp); c_2)$$

$$\textbf{while } sexp \textbf{ do } c \stackrel{\text{def}}{=} (\text{test } sexp; c)^*; \text{test } not(sexp)$$

$$\textbf{case } p \textbf{ of } p_1 \to c_1 | \cdots | p_n \to c_n | \stackrel{\text{def}}{=} (p_1 \text{ <?> } p; c_1) \oplus \cdots \oplus (p_n \text{ <?> } p; c_n)$$

Higher level strategies are defined using this syntactic sugar:

```
// iteratively applies a strategy s
//to the parameter x until it fails.

norm(s, x) =
(test s@x; x := s@x )*;
test not(s@x);
```

Strategy procedures

Commands can be given a name to be called later.

```
hrules = hf <+ hg
```

```
red(x) =
    if ?h(a,b)@x then
        x = try(hrules)
    else
        x := !g(x)
```

```
main(y) =
  red(y);
  if ?f(x)@y then
      x @= fg;
  else
      skip
```

Each call $strat(y)$ is executed in its own local scope. When returning to the caller, the final value of x is passed to the variable y.

Strategy procedures

Commands can be given a name to be called later.

```
hrules = hf <+ hg
```

```
red(x) =
    if ?h(a,b)@x then
        x = try(hrules)
    else
        x := !g(x)
```

```
main(y) =
  red(y);
  if ?f(x)@y then
      x @= fg;
  else
      skip
```

Each call $strat(y)$ is executed in its own local scope. When returning to the caller, the final value of x is passed to the variable y.

Recursive calls are allowed.

```
reduce = addZero <+ addSucc
reduceTD(x) =
  x @= try(reduce);
  case x of
  | add(a,b) -> reduceTD(a); reduceTD(b); x := !add(a,b); reduceTD(x)
  | succ(m) -> reduceTD(m); x := !succ(m)
  | 0 -> skip
```

Abstract Interpretation

Simulate a program semantics, keeping track only of relevant properties and **abstracting** from the concrete behavior.

C	concrete domain	A	abstract domain
$f : C \to C$	concrete semantics	$f^{\#} : A \to A$	abstract semantics
$\alpha : C \to A$	abstraction	$\gamma \ : A \to C$	concretization

Abstract Interpretation

Simulate a program semantics, keeping track only of relevant properties and **abstracting** from the concrete behavior.

C	concrete domain	A	abstract domain
$f : C \to C$	concrete semantics	$f^{\#} : A \to A$	abstract semantics
$\alpha : C \to A$	abstraction	$\gamma \ : A \to C$	concretization

The abstract semantics **over-approximate** the concrete behavior, to ensure that proven properties actually holds.

Arithmetic expression	Sign
$C = \mathbb{Z}$	$A = \{neg, pos, zero, dk\}$
$-10 * (4 + 7)$	neg $\otimes$ (pos $\oplus$ pos)
$-10 * (11)$	neg $\otimes$ pos
-110	neg

Local Completeness

Over-approximation introduces **false alerts**, invalid abstract executions that do not represent any concrete execution.

An ideal situation would be to have a **complete** abstract domain A.

Completeness: given the program f, for any concrete input $c \in C$ the abstraction A does not produce false alerts. Formally, completeness is stated as follows:

$$Af = AfA$$

where $A = \gamma \circ \alpha$.
Due to its **global notion**, completeness rarely holds.

Local Completeness

Over-approximation introduces **false alerts**, invalid abstract executions that do not represent any concrete execution.

An ideal situation would be to have a **complete** abstract domain A.

Completeness: given the program f, for any concrete input $c \in C$ the abstraction A does not produce false alerts. Formally, completeness is stated as follows:

$$Af = AfA$$

where $A = \gamma \circ \alpha$.
Due to its **global notion**, completeness rarely holds.

Local Completeness: relax the notion, consider only a smaller, fixed set of inputs.

$$\mathbb{C}_c^A \stackrel{\text{def}}{=} Af(c) = AfA(c)$$

Easier to prove, wider applicability in practice.

Local Completeness Logic [Bruni et al., 2021]

Idea: exploit local completeness to define a proof system, parametric to an abstraction A, capable of proving properties of a program without raising false alerts.

$$\vdash_A [P]c[Q]$$

Local Completeness Logic [Bruni et al., 2021]

Idea: exploit local completeness to define a proof system, parametric to an abstraction A, capable of proving properties of a program without raising false alerts.

$$\vdash_A [P]c[Q]$$

1. Q is an *under-approximation* of $[\![c]\!]P$.
2. $[\![c]\!]P$ is **locally complete** for input P on A.
3. Q and $[\![c]\!]P$ share the same **over-approximation** in A.

Local Completeness Logic [Bruni et al., 2021]

Idea: exploit local completeness to define a proof system, parametric to an abstraction A, capable of proving properties of a program without raising false alerts.

$$\vdash_A [P]c[Q]$$

1. Q is an *under-approximation* of $[\![c]\!]P$.
2. $[\![c]\!]P$ is **locally complete** for input P on A.
3. Q and $[\![c]\!]P$ share the same **over-approximation** in A.

Because of (1) and (2), any alert raised in Q is a **true alert**.

Local Completeness Logic [Bruni et al., 2021]

Idea: exploit local completeness to define a proof system, parametric to an abstraction A, capable of proving properties of a program without raising false alerts.

$$\vdash_A [P]c[Q]$$

1. Q is an *under-approximation* of $[\![c]\!]P$.
2. $[\![c]\!]P$ is **locally complete** for input P on A.
3. Q and $[\![c]\!]P$ share the same **over-approximation** in A.

Because of (1) and (2), any alert raised in Q is a **true alert**.

Because of (3), any proven property (expressible in A) is **guaranteed** to hold in the concrete execution.

The rules

$$\frac{\mathbb{C}_P^A(e)}{\vdash_A [P]e[[\![e]\!]P]} \text{ (transfer)}$$

$$\frac{P' \leq P \leq A(P') \quad \vdash_A [P']c[Q'] \quad Q \leq Q' \leq A(Q)}{\vdash_A [P]c[Q]} \text{ (relax)}$$

$$\frac{\vdash_A [P]c_1[R] \quad \vdash_A [R]c_2[Q]}{\vdash_A [P]c_1; c_2[Q]} \text{ (seq)}$$

$$\frac{\vdash [P]c_1[Q_1] \quad \vdash [P]c_2[Q_2]}{\vdash_A [P]c_1 \oplus c_2[Q_1 \vee Q_2]} \text{ (join)}$$

$$\frac{\vdash_A [P]c[R] \quad \vdash_A [P \vee R]c^*[Q]}{\vdash_A [P]c^*[Q]} \text{ (rec)}$$

$$\frac{\vdash_A [P]c[Q] \quad Q \leq A(P)}{\vdash_A [P]c^*[P \vee Q]} \text{ (iterate)}$$

In the original proof system, (transfer) is parametric to a generic set of expressions $e \in Exp$.

In **Strat**, the basic expressions become $x := sexp$, $sexp$ <?> p, and **test** $sexp$.

The rules

$$\frac{\mathbb{C}_P^A(e)}{\vdash_A [P]e[[\![e]\!]P]} \text{ (transfer)} \qquad \frac{P' \leq P \leq A(P') \quad \vdash_A [P']c[Q'] \quad Q \leq Q' \leq A(Q)}{\vdash_A [P]c[Q]} \text{ (relax)}$$

$$\frac{\vdash_A [P]c_1[R] \quad \vdash_A [R]c_2[Q]}{\vdash_A [P]c_1; c_2[Q]} \text{ (seq)} \qquad \frac{\vdash [P]c_1[Q_1] \quad \vdash [P]c_2[Q_2]}{\vdash_A [P]c_1 \oplus c_2[Q_1 \vee Q_2]} \text{ (join)}$$

$$\frac{\vdash_A [P]c[R] \quad \vdash_A [P \vee R]c^*[Q]}{\vdash_A [P]c^*[Q]} \text{ (rec)} \qquad \frac{\vdash_A [P]c[Q] \quad Q \leq A(P)}{\vdash_A [P]c^*[P \vee Q]} \text{ (iterate)}$$

$$\frac{strat(x) = s \in Defs \quad \vdash_A [P[x/y][\overline{z}/\overline{w}]]s[Q]}{\vdash_A [P]strat(y)[Q[y/x][\overline{w}/\overline{z}]]} \ (call)$$

In the original proof system, (transfer) is parametric to a generic set of expressions $e \in Exp$.

In **Strat**, the basic expressions become $x := sexp$, $sexp$ <?> p, and **test** $sexp$.

The (call) rule was added to handle procedure calls.

Even program

$$\Sigma = \{0, succ, even, true, false\}$$

Rules:

$$\begin{aligned} \text{evenZero} &: even(0) \to true \\ \text{evenSuccZero} &: even(succ(0)) \to false \\ \text{evenSucc} &: even(succ(succ(x))) \to even(x) \end{aligned}$$

The program:

```
evenN = evenZero <+ evenSuccZero <+ evenSucc

norm(evenN, n) =
  while evenN@n  do
    n @= evenN;
```

Even program

$$\Sigma = \{0, succ, even, true, false\}$$

Rules:

$$\begin{aligned} \text{evenZero} &: even(0) \to true \\ \text{evenSuccZero} &: even(succ(0)) \to false \\ \text{evenSucc} &: even(succ(succ(x))) \to even(x) \end{aligned}$$

The program:

```
evenN = evenZero <+ evenSuccZero <+ evenSucc

norm(evenN, n) =
  (test even@N; n @= evenN)*;
  test not(evenN)@N
```

Even program

$$\Sigma = \{0, succ, even, true, false\}$$

Rules:

$$\begin{aligned} \text{evenZero} &: even(0) \to true \\ \text{evenSuccZero} &: even(succ(0)) \to false \\ \text{evenSucc} &: even(succ(succ(x))) \to even(x) \end{aligned}$$

The program:

```
evenN = evenZero <+ evenSuccZero <+ evenSucc

norm(evenN, n) =
  (test even@N; n @= evenN)*;
  test not(evenN)@N
```

Property: When n is even, norm(evenN, n) rewrites it to true.

$$Even = \{0, succ(succ(0)), ...\} \qquad Odd = \{succ(0), succ(succ(succ(0))), ...\}$$

Even program

$$\Sigma = \{0, succ, even, true, false\}$$

Rules:

$$\begin{aligned} \text{evenZero} &: even(0) \to true \\ \text{evenSuccZero} &: even(succ(0)) \to false \\ \text{evenSucc} &: even(succ(succ(x))) \to even(x) \end{aligned}$$

The program:

```
evenN = evenZero <+ evenSuccZero <+ evenSucc

norm(evenN, n) =
  (test even@N; n @= evenN)*;
  test not(evenN)@N
```

Property: When n is even, norm(evenN, n) rewrites it to true.

$$Even = \{0, succ(succ(0)), ...\} \qquad Odd = \{succ(0), succ(succ(succ(0))), ...\}$$

$$\vdash_A [\{even(n) \mid n \in Even\}]norm(evenN, n)[\{true\}]$$

Term abstraction

Define a set $Shapes$ to abstract terms based on their structure.

A set of terms $\mathbf{T}$ will be abstracted by the corresponding set of shapes $\alpha(\mathbf{T})$.

$$Shapes = \{e,\ o, tt, \mathit{ff}, \mathit{fail}, a\}$$

Term abstraction

Define a set $Shapes$ to abstract terms based on their structure.

A set of terms $\mathbf{T}$ will be abstracted by the corresponding set of shapes $\alpha(\mathbf{T})$.

$$Shapes = \{e,\ o, tt, \mathit{ff}, \mathit{fail}, a\}$$

Concretization γ:

- $e \mapsto \{even(n) \mid n \in Even\}$
- $o \mapsto \{even(n) \mid n \in Odd\}$
- $tt \mapsto true, mathitff \mapsto false, \mathit{fail} \mapsto KO$
- Any other term is mapped to the generic shape a.

Term abstraction

Define a set $Shapes$ to abstract terms based on their structure.

A set of terms $\mathbf{T}$ will be abstracted by the corresponding set of shapes $\alpha(\mathbf{T})$.

$$Shapes = \{e,\ o, tt, \mathit{ff}, \mathit{fail}, a\}$$

Concretization γ:

- $e \mapsto \{even(n) \mid n \in Even\}$
- $o \mapsto \{even(n) \mid n \in Odd\}$
- $tt \mapsto true, mathitff \mapsto false, \mathit{fail} \mapsto KO$
- Any other term is mapped to the generic shape a.

$$T = \{even(succ(0)), false\} \qquad \alpha(T) = \{o, \mathit{ff}\}$$

Term abstraction

Define a set $Shapes$ to abstract terms based on their structure.

A set of terms $\mathbf{T}$ will be abstracted by the corresponding set of shapes $\alpha(\mathbf{T})$.

$$Shapes = \{e,\ o, tt, \mathit{ff}, \mathit{fail}, a\}$$

Concretization γ:

- $e \mapsto \{even(n) \mid n \in Even\}$
- $o \mapsto \{even(n) \mid n \in Odd\}$
- $tt \mapsto true, mathitff \mapsto false, \mathit{fail} \mapsto KO$
- Any other term is mapped to the generic shape a.

$$T = \{even(succ(0)), false\} \qquad \alpha(T) = \{o, \mathit{ff}\}$$

$$T = \{even(succ(succ(0))), true, succ(false)\} \qquad \alpha(T) = \{e, tt, a\}$$

A proof

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P_2}(c_{11})}{[P_2]\ c_{11}\ [\{even(0)\}]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^A_{\{0\}}(c_{12})}{[\{even(0)\}]\ c_{12}[P_3]}\ \text{(assign)}}{\vdash_A\ [P_2]\ c_1\ [P_3]}\ \text{(seq)} \qquad P_3 \subseteq A(P_2) = \{tt, e\}}{(**)}\ \text{(iterate)}$$

$$\dfrac{P_2 \subseteq P_1 \subseteq A(P_2) = \{tt, e\} \qquad \dfrac{(**)}{\vdash_A\ [P_2]\ c_1\ [P_2]}\ \text{(iterate)} \qquad P_2 \subseteq P_2 \subseteq A(P_2) = \{tt, e\}}{(*)}\ \text{(relax)}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathbb{C}^A_{P}(c_{11})}{\vdash_A\ [P]\ c_{11}\ [P]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^A_{P}(c_{12})}{\vdash_A\ [P]\ c_{12}\ [P_1]}\ \text{(assign)}}{\vdash_A\ [P]\ c_1\ [P_1]}\ \text{(seq)} \qquad \dfrac{(*)}{\vdash_A\ [P_1]\ c_1^*\ [P_2]}\ \text{(relax)}}{\vdash_A\ [P]\ c_1^*\ [P_2]}\ \text{(rec)} \qquad \dfrac{\mathbb{C}^A_{P_2}(c_2)}{\vdash_A\ [P_2]\ c_2\ [P_3]}\ \text{(test)}}{\vdash_A\ [P]\ \text{norm(evenN,n)}\ [P_3]}\ \text{(seq)}$$

$c_1 \triangleq (\text{test}\ evenN@n; n@ = evenN)^*$

$c_2 \triangleq \text{test not(evenN)@n}$

$c_{11} \triangleq \text{test}\ evenN@n$

$c_{12} \triangleq n@ = evenN$

$P = \{even(n) \mid n \in Even\}$

$P_1 \triangleq \{true\} \cup P$

$P_2 \triangleq \{true, even(0)\}$

$P_3 \triangleq \{true\}$

A proof

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^{A}_{P_2}(c_{11})}{[P_2]\ c_{11}\ [\{even(0)\}]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^{A}_{\{0\}}(c_{12})}{[\{even(0)\}]\ c_{12}[P_3]}\ \text{(assign)}}{\vdash_A\ [P_2]\ c_1\ [P_3]}\ \text{(seq)} \qquad P_3 \subseteq A(P_2) = \{tt, e\}}{(**)}\ \text{(iterate)}$$

$$\dfrac{P_2 \subseteq P_1 \subseteq A(P_2) = \{tt, e\} \qquad \dfrac{(**)}{\vdash_A\ [P_2]\ c_1\ [P_2]}\ \text{(iterate)} \qquad P_2 \subseteq P_2 \subseteq A(P_2) = \{tt, e\}}{(*)}\ \text{(relax)}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathbb{C}^{A}_{P}(c_{11})}{\vdash_A\ [P]\ c_{11}\ [P]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^{A}_{P}(c_{12})}{\vdash_A\ [P]\ c_{12}\ [P_1]}\ \text{(assign)}}{\vdash_A\ [P]\ c_1\ [P_1]}\ \text{(seq)} \qquad \dfrac{(*)}{\vdash_A\ [P_1]\ c_1^*\ [P_2]}\ \text{(relax)}}{\vdash_A\ [P]\ c_1^*\ [P_2]}\ \text{(rec)} \qquad \dfrac{\mathbb{C}^{A}_{P_2}(c_2)}{\vdash_A\ [P_2]\ c_2\ [P_3]}\ \text{(test)}}{\vdash_A\ [P]\ \text{norm(evenN,n)}\ [P_3]}\ \text{(seq)}$$

$c_1 \triangleq (\text{test}\ evenN@n; n@ = evenN)^*$

$c_2 \triangleq \text{test not(evenN)@n}$

$c_{11} \triangleq \text{test}\ evenN@n$

$c_{12} \triangleq n@ = evenN$

$P = \{even(n) \mid n \in Even\}$

$P_1 \triangleq \{true\} \cup P$

$P_2 \triangleq \{true, even(0)\}$

$P_3 \triangleq \{true\}$

A proof

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^{A}_{P_2}(c_{11})}{[P_2]\ c_{11}\ [\{even(0)\}]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^{A}_{\{0\}}(c_{12})}{[\{even(0)\}]\ c_{12}[P_3]}\ \text{(assign)}}{\vdash_A\ [P_2]\ c_1\ [P_3]}\ \text{(seq)} \qquad P_3 \subseteq A(P_2) = \{tt, e\}}{(**)}\ \text{(iterate)}$$

$$\dfrac{P_2 \subseteq P_1 \subseteq A(P_2) = \{tt, e\} \qquad \dfrac{(**)}{\vdash_A\ [P_2]\ c_1\ [P_2]}\ \text{(iterate)} \qquad P_2 \subseteq P_2 \subseteq A(P_2) = \{tt, e\}}{(*)}\ \text{(relax)}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathbb{C}^{A}_{P}(c_{11})}{\vdash_A\ [P]\ c_{11}\ [P]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^{A}_{P}(c_{12})}{\vdash_A\ [P]\ c_{12}\ [P_1]}\ \text{(assign)}}{\vdash_A\ [P]\ c_1\ [P_1]}\ \text{(seq)} \qquad \dfrac{(*)}{\vdash_A\ [P_1]\ c_1^*\ [P_2]}\ \text{(relax)}}{\vdash_A\ [P]\ c_1^*\ [P_2]}\ \text{(rec)} \qquad \dfrac{\mathbb{C}^{A}_{P_2}(c_2)}{\vdash_A\ [P_2]\ c_2\ [P_3]}\ \text{(test)}}{\vdash_A\ [P]\ \text{norm(evenN,n)}\ [P_3]}\ \text{(seq)}$$

$c_1 \triangleq (\text{test}\ evenN@n; n@ = evenN)^*$

$c_2 \triangleq \text{test not(evenN)@n}$

$c_{11} \triangleq \text{test}\ evenN@n$

$c_{12} \triangleq n@ = evenN$

$P = \{even(n) \mid n \in Even\}$

$P_1 \triangleq \{true\} \cup P$

$P_2 \triangleq \{true, even(0)\}$

$P_3 \triangleq \{true\}$

A proof

$$\dfrac{\dfrac{\dfrac{\mathbb{C}^{A}_{P_2}(c_{11})}{[P_2]\ c_{11}\ [\{even(0)\}]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^{A}_{\{0\}}(c_{12})}{[\{even(0)\}]\ c_{12}[P_3]}\ \text{(assign)}}{\vdash_A\ [P_2]\ c_1\ [P_3]}\ \text{(seq)} \qquad P_3 \subseteq A(P_2) = \{tt, e\}}{(**)}\ \text{(iterate)}$$

$$\dfrac{P_2 \subseteq P_1 \subseteq A(P_2) = \{tt, e\} \qquad \dfrac{(**)}{\vdash_A\ [P_2]\ c_1\ [P_2]}\ \text{(iterate)} \qquad P_2 \subseteq P_2 \subseteq A(P_2) = \{tt, e\}}{(*)}\ \text{(relax)}$$

$$\dfrac{\dfrac{\dfrac{\dfrac{\mathbb{C}^{A}_{P}(c_{11})}{\vdash_A\ [P]\ c_{11}\ [P]}\ \text{(test)} \qquad \dfrac{\mathbb{C}^{A}_{P}(c_{12})}{\vdash_A\ [P]\ c_{12}\ [P_1]}\ \text{(assign)}}{\vdash_A\ [P]\ c_1\ [P_1]}\ \text{(seq)} \qquad \dfrac{(*)}{\vdash_A\ [P_1]\ c_1^*\ [P_2]}\ \text{(relax)}}{\vdash_A\ [P]\ c_1^*\ [P_2]}\ \text{(rec)} \qquad \dfrac{\mathbb{C}^{A}_{P_2}(c_2)}{\vdash_A\ [P_2]\ c_2\ [P_3]}\ \text{(test)}}{\vdash_A\ [P]\ \text{norm(evenN,n)}\ [P_3]}\ \text{(seq)}$$

$c_1 \triangleq (\text{test}\ evenN@n; n@ = evenN)^*$

$c_2 \triangleq \text{test not(evenN)@n}$

$c_{11} \triangleq \text{test}\ evenN@n$

$c_{12} \triangleq n@ = evenN$

$P = \{even(n) \mid n \in Even\}$

$P_1 \triangleq \{true\} \cup P$

$P_2 \triangleq \{true, even(0)\}$

$P_3 \triangleq \{true\}$

Bug detection

What happens if we also consider $even(succ(0))$ in the initial input?

$$\vdash_A [\{even(n) \mid n \in Even\} \cup \{even(succ(0))\}]norm(evenN, n)[\{true, false\}]$$

The property $n = true$ is violated, and the analysis reports the true alert $false$.

Conclusions

In this work, we have presented the novel strategy language ***Strat*** *and defined a verification framework for its programs, based on* ***Local Completeness Logic****.*

Conclusions

In this work, we have presented the novel strategy language ***Strat*** *and defined a verification framework for its programs, based on* ***Local Completeness Logic****.*

- Future work:
  - **Strat** features: **sorts, conditional rules, narrowing**, etc.
  - Research **tree automata** [Comon et al., 2008] as a possible abstract domain.
  - LCL on existing strategy languages: **Stratego, Maude, Elan**, ...

Conclusions

In this work, we have presented the novel strategy language ***Strat*** *and defined a verification framework for its programs, based on* ***Local Completeness Logic****.*

- Future work:
  - **Strat** features: **sorts, conditional rules, narrowing**, etc.
  - Research **tree automata** [Comon et al., 2008] as a possible abstract domain.
  - LCL on existing strategy languages: **Stratego, Maude, Elan**, ...
- Related work:
  - Regular language type inference on term rewriting systems. [Haudebourg et al., 2020]
  - Reachability analysis by tree automata completion. [Feuillade et al., 2004]
  - Abstract interpretation of Stratego. [Keidel and Erdweg, 2017]

Thank you for your attention!